École Supérieure Privée d'Ingénierie et de Technologies

Module AWD – MT-41 · Applications Web Distribuées

Dossier technique

Validation intermédiaire – Semaine 4

MedFinder

Trouver, réserver et être alerté de la disponibilité
d'un médicament dans les pharmacies proches

Réalisé par :
Jihed Khlifi
Abdelhamid Ayadi
Ferdaouas Oueslati
Insaf Makhloufi

Groupe : Gen Z
Classe : 4DASI11
Encadré par : Mme Ons Ben Salah

Année universitaire 2026 – 2027

Table des matières

1 Description du sujet **4**

1.1 Contexte 4

1.2 Problématique 4

1.3 Solution proposée 4

1.4 Objectifs techniques 4

1.5 Périmètre 5

2 Analyse fonctionnelle **6**

2.1 Acteurs 6

2.2 Cas d'utilisation 6

2.3 Fonctionnalités (récits utilisateurs) 6

2.4 Règles métier 7

2.5 Cycle de vie d'une réservation 8

2.6 Besoins non fonctionnels 8

3 Découpage en microservices **9**

3.1 Démarche 9

3.2 Les cinq microservices 9

3.3 Justification du découpage 9

3.4 Choix des bases de données 10

4 Architecture technique **11**

4.1 Architecture globale 11

4.2 Composants d'infrastructure 12

4.3 Technologies 12

4.4 Sécurité 12

5 Données et communication **14**

5.1 Propriété des données 14

5.2 Communication synchrone 14

5.3 Communication asynchrone 14

5.4 Scénario synchrone : réserver un médicament 15

5.5 Scénario asynchrone : alerte de retour en stock 16

5.6 Cohérence et gestion des pannes 16

5.7 Conventions des API 16

6 Conception détaillée des microservices **18**
6.1 MS1 · Pharmacies . . . 18
6.1.1 Responsabilités . . . 18
6.1.2 Diagramme de classes . . . 18
6.1.3 Interactions . . . 18
6.1.4 Contrat d'API . . . 19
6.2 MS2 · Catalogue des médicaments . . . 19
6.2.1 Responsabilités . . . 19
6.2.2 Diagramme de classes . . . 20
6.2.3 Interactions . . . 20
6.2.4 Contrat d'API . . . 21
6.3 MS3 · Stocks . . . 21
6.3.1 Responsabilités . . . 21
6.3.2 Diagramme de classes . . . 21
6.3.3 Interactions . . . 22
6.3.4 Contrat d'API . . . 22
6.4 MS4 · Réservations et alertes . . . 22
6.4.1 Responsabilités . . . 23
6.4.2 Diagramme de classes . . . 23
6.4.3 Interactions . . . 23
6.4.4 Contrat d'API . . . 24
6.5 MS5 · Notifications et statistiques (microservice commun) . . . 24
6.5.1 Responsabilités . . . 24
6.5.2 Diagramme de classes . . . 25
6.5.3 Interactions . . . 25
6.5.4 Contrat d'API . . . 25

7 Organisation du projet **26**
7.1 Répartition des tâches . . . 26
7.2 Dépôt Git . . . 26
7.3 Gestion de projet . . . 27
7.4 Planning . . . 27
7.5 Risques identifiés . . . 27

Conclusion **28**

A Exemples d'échanges **29**
A.1 Créer une réservation . . . 29
A.2 Événement `stock.replenished` . . . 29
A.3 Événement `reservation.released` . . . 29

Table des figures

2.1 Diagramme de cas d'utilisation 6
2.2 États d'une réservation 8

4.1 Architecture globale de MedFinder 11

5.1 Diagramme de séquence de la réservation 15
5.2 Diagramme de séquence de l'alerte 16

6.1 MS1 – diagramme de classes 18
6.2 MS1 – interactions 18
6.3 MS2 – diagramme de classes 20
6.4 MS2 – interactions 20
6.5 MS3 – diagramme de classes 21
6.6 MS3 – interactions 22
6.7 MS4 – diagramme de classes 23
6.8 MS4 – interactions 23
6.9 MS5 – documents MongoDB 25
6.10 MS5 – interactions 25

CHAPITRE 1

Description du sujet

1.1 Contexte

Chercher un médicament en rupture est une expérience courante : le patient se déplace de pharmacie en pharmacie ou téléphone à plusieurs officines sans savoir laquelle dispose du produit. Les pharmaciens, de leur côté, reçoivent de nombreux appels pour des questions de disponibilité et n'ont pas d'outil simple pour informer les patients du retour d'un produit.

1.2 Problématique

Comment permettre à un patient de savoir, en quelques secondes, **quelle pharmacie proche dispose d'un médicament**, de **le réserver sans risque de survente**, et d'**être prévenu automatiquement** lorsqu'un médicament indisponible revient en stock, tout en laissant chaque pharmacie gérer ses propres stocks ?

1.3 Solution proposée

MedFinder est une application web distribuée, construite en microservices, qui relie patients et pharmacies :

- **Rechercher** un médicament (par nom ou molécule active) et voir les pharmacies qui l'ont en stock, avec le prix et la distance ;
- **Réserver** des unités dans une pharmacie et recevoir un code de retrait ;
- **Créer une alerte** sur un médicament indisponible et être notifié dès son réapprovisionnement ;
- **Gérer** pour le pharmacien : sa pharmacie, ses stocks, ses réservations ;
- **Superviser** pour l'administrateur : validation des pharmacies, catalogue, statistiques.

1.4 Objectifs techniques

- Découper le domaine en cinq microservices autonomes, chacun propriétaire de ses données ;
- Mettre en place l'infrastructure Spring Cloud : API Gateway, Eureka, Config Server ;
- Sécuriser l'ensemble avec Keycloak (OpenID Connect, jetons JWT, rôles) ;
- Combiner communication synchrone (OpenFeign, avec circuit breaker) et asynchrone (RabbitMQ) ;
- Conteneuriser et lancer toute la plateforme avec Docker Compose.

1.5 Périmètre

Inclus : recherche, réservation, alertes, notifications dans l'application, gestion des stocks et des pharmacies, statistiques. Hors périmètre : paiement en ligne, livraison à domicile, téléversement et vérification d'ordonnances, notifications SMS ou e-mail réelles (simulées par des notifications dans l'application).

CHAPITRE 2

Analyse fonctionnelle

2.1 Acteurs

Acteur	Description	Rôle Keycloak
Visiteur	Utilisateur non connecté : recherche de médicaments et de pharmacies	-
Patient	Réserve des médicaments, crée des alertes, consulte ses notifications	`PATIENT`
Pharmacien	Gère sa pharmacie, ses stocks et les réservations reçues	`PHARMACIST`
Administrateur	Valide les pharmacies, gère le catalogue, consulte les statistiques	`ADMIN`

Table 2.1 – Acteurs du système

2.2 Cas d'utilisation

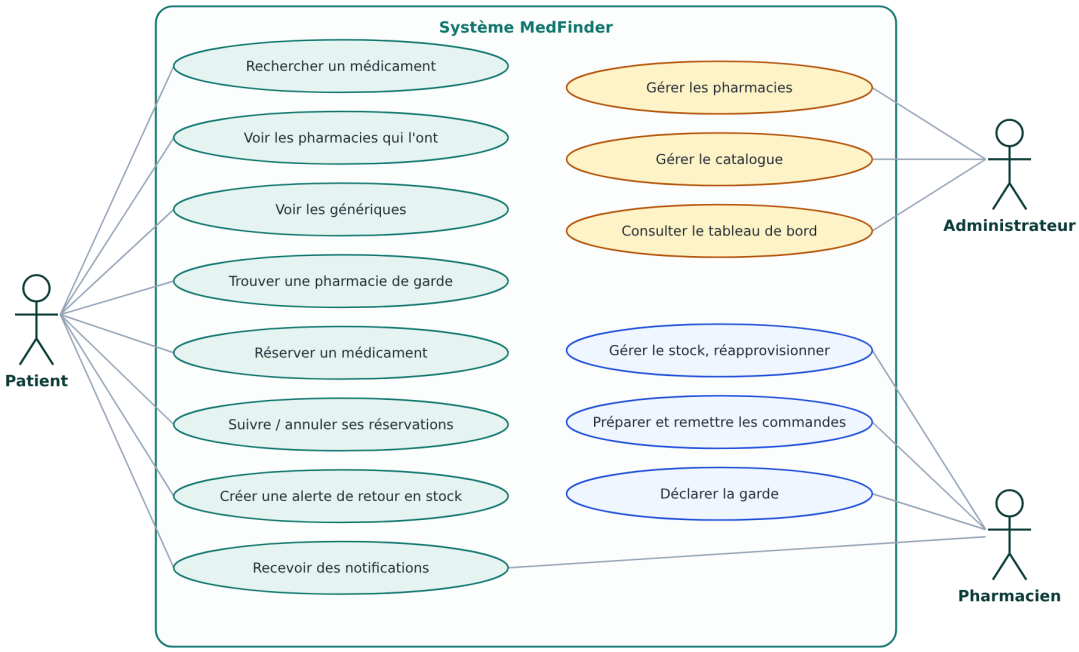


Figure 2.1 – Diagramme de cas d'utilisation

2.3 Fonctionnalités (récits utilisateurs)

ID	Acteur	Récit	MS
US1	Visiteur	Rechercher un médicament par nom ou molécule active	MS2
US2	Visiteur	Voir les pharmacies qui ont un médicament en stock, avec prix et quantité	MS3
US3	Visiteur	Trouver les pharmacies proches et les pharmacies de garde	MS1
US4	Visiteur	Consulter les génériques d'un médicament (même molécule)	MS2
US5	Patient	Réserver un médicament et recevoir un code de retrait	MS4
US6	Patient	Suivre et annuler ses réservations	MS4
US7	Patient	Créer une alerte de retour en stock et la supprimer	MS4
US8	Patient	Consulter ses notifications et les marquer comme lues	MS5
US9	Pharmacien	Créer et modifier la fiche de sa pharmacie (horaires, garde)	MS1
US10	Pharmacien	Ajouter un médicament à son stock, ajuster quantité et prix	MS3
US11	Pharmacien	Réapprovisionner un stock et consulter l'historique des mouvements	MS3
US12	Pharmacien	Marquer une réservation « prête » puis « retirée »	MS4
US13	Administrateur	Valider ou désactiver une pharmacie	MS1
US14	Administrateur	Gérer le catalogue des médicaments et des catégories	MS2
US15	Administrateur	Consulter le journal des événements et les statistiques	MS5

Table 2.2 – Récits utilisateurs et microservice responsable

2.4 Règles métier

RG1 Une réservation ne peut jamais dépasser la quantité disponible : deux patients ne peuvent pas obtenir la dernière boîte (décrément atomique).

RG2 Une réservation non retirée expire automatiquement après 24 heures ; les unités sont rendues au stock.

RG3 Chaque réservation possède un code de retrait unique, présenté au pharmacien.

RG4 Un patient ne peut avoir qu'une seule alerte active par médicament (sinon `409 Conflict`).

RG5 Un pharmacien ne gère que la pharmacie et les stocks qui lui sont rattachés (`403` sinon).

RG6 Une pharmacie créée n'est visible des patients qu'après validation par l'administrateur.

RG7 Un médicament soumis à ordonnance peut être réservé ; l'ordonnance est présentée au retrait.

RG8 Une alerte est déclenchée lorsqu'un stock d'un médicament repasse au-dessus de zéro.

2.5 Cycle de vie d'une réservation

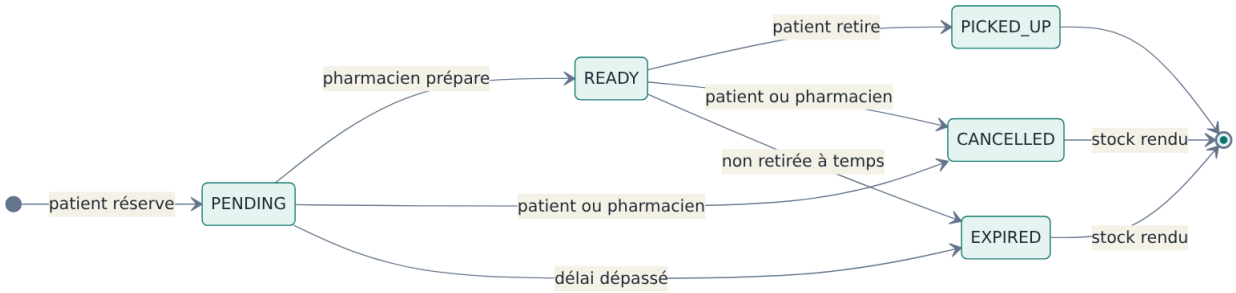


Figure 2.2 – États d'une réservation

2.6 Besoins non fonctionnels

Besoin	Réponse prévue
Sécurité	Authentification Keycloak (OIDC), jetons JWT vérifiés par la Gateway et par chaque service, contrôle par rôle
Disponibilité	Circuit breaker Resilience4j, mode dégradé, files RabbitMQ durables
Cohérence	Décrément atomique en SQL, compensation par événement, cohérence à terme pour les notifications
Évolutivité	Services indépendants, découverte dynamique (Eureka), configuration centralisée
Portabilité	Une image Docker par service, lancement complet par Docker Compose
Documentation	Contrats OpenAPI et Swagger UI pour chaque microservice
Traçabilité	Journal de tous les événements métier dans MS5

CHAPITRE 3

Découpage en microservices

3.1 Démarche

Le découpage suit les **contextes bornés** du domaine : chaque microservice correspond à une capacité métier cohérente, possède ses propres données (*database per service*) et peut évoluer, être déployé et tomber en panne indépendamment des autres. Les références entre services se font par identifiant (`pharmacyId`, `medicineId`) et jamais par jointure entre bases.

3.2 Les cinq microservices

Microservice	Responsabilité	Base	Responsable
MS1 Pharmacies	Pharmacies, horaires, position, gardes, validation	H2	Jihed Khlifi
MS2 Médicaments	Catalogue, catégories, recherche, génériques	H2	Abdelhamid Ayadi
MS3 Stocks	Stock par pharmacie, prix, mouvements, réservation d'unités	H2	Ferdaouas Oueslati
MS4 Réservations	Réservations, codes de retrait, expiration, alertes	MySQL	Insaf Makhloufi
MS5 Notifications (commun)	Notifications, journal des événements, statistiques	MongoDB	Toute l'équipe

Table 3.1 – Découpage et responsabilités

3.3 Justification du découpage

— **MS1 et MS2 sont des services de référence** : données surtout lues, qui changent rarement, utilisées par MS3 et MS4. Les séparer évite qu'une modification du catalogue impacte la gestion des pharmacies.

— **MS3 isole la donnée la plus sensible**, la quantité en stock, afin de garantir l'absence de survente en un seul endroit.

— **MS4 porte le parcours patient** (réservation et alerte) : il orchestre les appels nécessaires et produit les événements métier.

— **MS5 est transverse** : il n'appelle aucun service, il écoute les événements. Il peut être arrêté sans bloquer la réservation.

3.4 Choix des bases de données

Service	Base	Justification
MS1	H2	Imposée pour les microservices individuels ; données de référence simples, volume faible
MS2	H2	Imposée ; catalogue relationnel (catégorie 1–n médicament), surtout lu
MS3	H2	Imposée ; les mises à jour atomiques en SQL garantissent l'absence de survente
MS4	MySQL	Réservations transactionnelles qui doivent survivre à un redémarrage ; base persistante dans son conteneur
MS5	MongoDB	Technologie avancée du service commun : chaque événement a une structure différente (documents), statistiques par pipeline d'agrégation

CHAPITRE 4

Architecture technique

4.1 Architecture globale

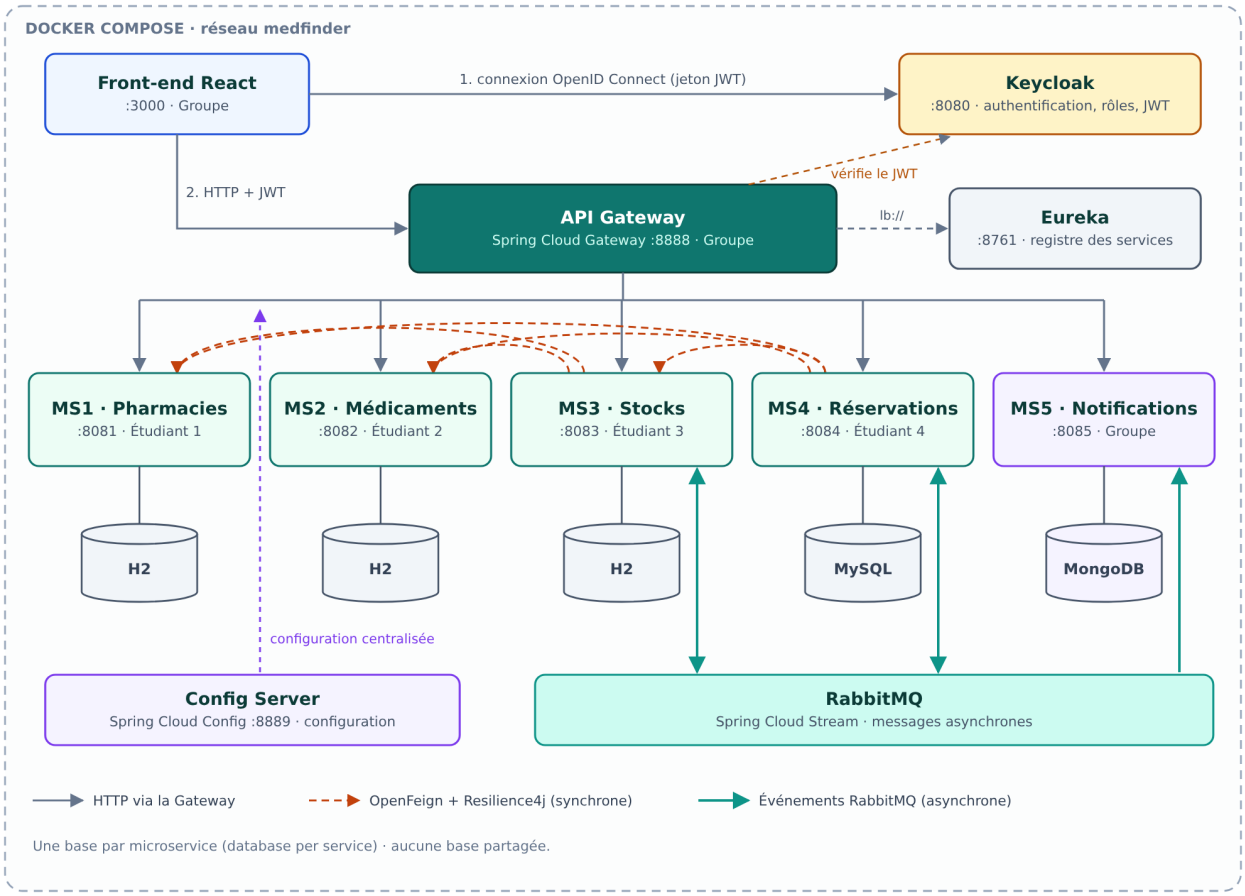


Figure 4.1 – Architecture globale de MedFinder

Le client React n'appelle qu'un seul point d'entrée, l'API Gateway. Celle-ci vérifie le jeton JWT délivré par Keycloak, puis route la requête vers le microservice concerné, dont elle obtient l'adresse auprès d'Eureka. Au démarrage, chaque service récupère sa configuration auprès du Config Server. Les échanges entre services sont synchrones (OpenFeign) lorsqu'une réponse est nécessaire pour continuer, et asynchrones (RabbitMQ) pour diffuser un fait métier.

4.2 Composants d'infrastructure

Composant	Rôle	Port
Front-end React	Interface patient, pharmacien et administrateur ; connexion via Keycloak	5173
API Gateway	Point d'entrée unique : routage `lb://`, validation JWT, CORS, routes internes bloquées	8080
Eureka Server	Registre : chaque service s'enregistre, la Gateway et Feign le découvrent par son nom	8761
Config Server	Configuration centralisée (ports, bases, RabbitMQ, Eureka) avec profils `dev` et `docker`	8888
Keycloak	Royaume `medfinder`, utilisateurs, rôles, émission des jetons	8180
RabbitMQ	Courtier de messages : échanges et files durables des événements	5672
MySQL / MongoDB	Bases de MS4 et de MS5, chacune dans son conteneur	3306 / 27017
MS1 à MS5	Microservices métier, documentés par Swagger UI	8081–8085

4.3 Technologies

Besoin	Technologie	Usage
Microservices	Spring Boot 3.3, Java 17, Maven	Un projet par microservice
Infrastructure	Spring Cloud 2023 (Gateway, Netflix Eureka, Config)	Routage, découverte, configuration
Sécurité	Keycloak 25, Spring Security OAuth2 Resource Server	Connexion, JWT, rôles
Synchrone	OpenFeign, Resilience4j	Appels REST entre services, circuit breaker
Asynchrone	RabbitMQ, Spring Cloud Stream	Publication et consommation d'événements
Persistance	Spring Data JPA (H2, MySQL), Spring Data MongoDB	Une base par service
Documentation	springdoc-openapi, Swagger UI	Contrats d'API
Front-end	React, Vite	Interface web
Déploiement	Docker, Docker Compose	Lancement de toute la plateforme
Gestion de projet	Git, GitHub, GitHub Projects	Versionnement, tâches, revues

4.4 Sécurité

— **Keycloak** gère les comptes et les rôles `PATIENT`, `PHARMACIST`, `ADMIN` dans le royaume `medfinder`, exporté dans le dépôt Git pour être réimporté automatiquement.

— Le front-end utilise le flux *Authorization Code* avec PKCE et envoie le jeton dans l'en-tête `Authorization: Bearer`.

— La **Gateway** valide la signature et l'expiration du jeton ; les lectures publiques (recherche) restent accessibles sans jeton.

— Chaque **microservice** valide aussi le jeton (défense en profondeur) et applique les règles fines (`@PreAuthorize`, propriétaire de la ressource).

— Les appels Feign propagent le jeton de l'utilisateur ; la route interne de MS3 (`/internal/...`) n'est pas exposée par la Gateway.

CHAPITRE 5

Données et communication

5.1 Propriété des données

Chaque microservice est le seul à lire et écrire dans sa base. Un autre service qui a besoin d'une information la demande par l'API (synchrone) ou la reçoit dans un événement (asynchrone). Pour éviter des appels répétés, MS3 conserve une copie du nom du médicament (`medicineName`) au moment de l'ajout au stock.

5.2 Communication synchrone

On choisit le synchrone lorsque l'appelant **a besoin de la réponse pour continuer**. Tous les appels passent par OpenFeign, avec résolution du nom par Eureka et un circuit breaker Resilience4j.

Appel	Endpoint	Pourquoi synchrone	Si l'appelé tombe
MS3 → MS1	`GET /pharmacies/{id}`	Afficher les pharmacies qui ont le médicament	Stocks renvoyés sans le détail des pharmacies (mode dégradé)
MS3 → MS2	`GET /medicines/{id}`	Vérifier que le médicament existe avant l'ajout au stock	Ajout refusé, `503` explicite
MS4 → MS2	`GET /medicines/{id}`	Connaître le nom et l'exigence d'ordonnance	Réservation refusée, `503`
MS4 → MS1	`GET /pharmacies/{id}`	Vérifier la pharmacie, afficher son nom	Réservation créée sans le nom (mode dégradé)
MS4 → MS3	`POST /stocks/internal/reservations`	Prendre les unités : la réponse décide de la création	Réservation refusée, `503`, rien n'est enregistré

Table 5.1 – Matrice des appels synchrones (préfixe `/api/v1`)

5.3 Communication asynchrone

On choisit l'asynchrone pour **diffuser un fait accompli** à des services qui n'ont pas à bloquer l'émetteur. Les messages restent dans des files durables RabbitMQ si le consommateur est arrêté.

Événement	Émetteur	Reçu par	Déclencheur et contenu
`stock.replenished`	MS3	MS4, MS5	Réapprovisionnement d'un stock. `stockId, pharmacyId, medicineId, medicineName, quantity, occurredAt`

Événement	Émetteur	Reçu par	Déclencheur et contenu
`reservation.events`	MS4	MS5	Création ou changement d'état d'une réservation. `reservationId, type, patientUsername, pharmacyId, medicineId, quantity, pickupCode`
`alert.triggered`	MS4	MS5	Alerte déclenchée par un retour en stock. `alertId, patientUsername, medicineId, pharmacyId`
`reservation.released`	MS4	MS3	Réservation annulée ou expirée : les unités doivent être rendues. `reservationId, pharmacyId, medicineId, quantity`

Table 5.2 – Événements métier

5.4 Scénario synchrone : réserver un médicament

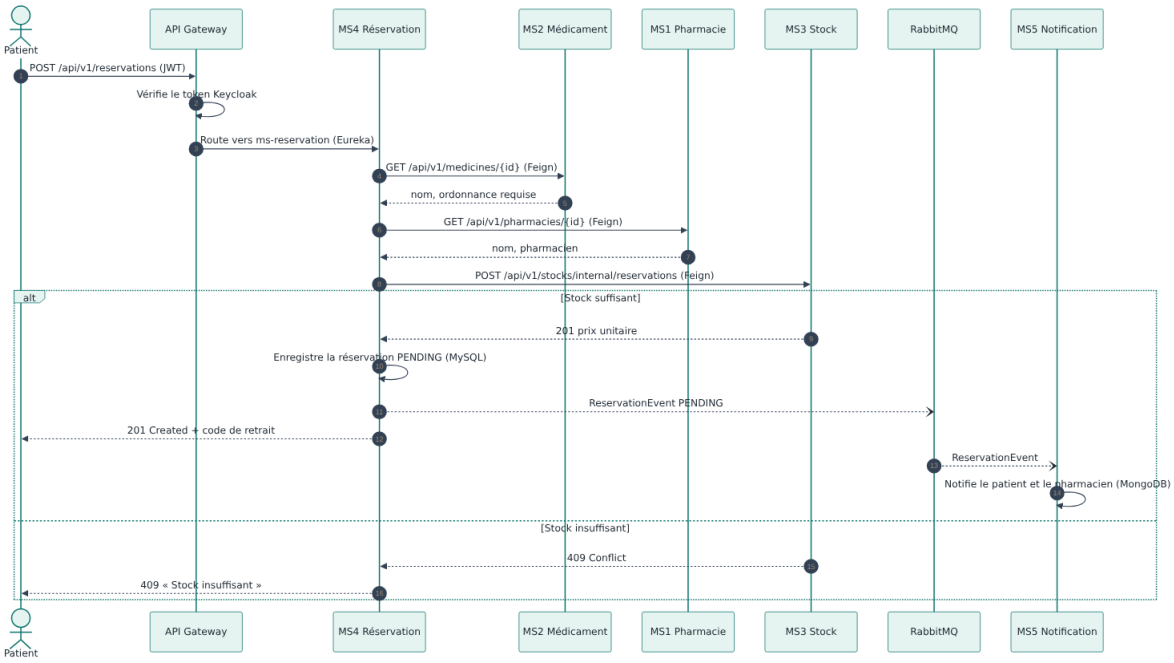


Figure 5.1 – Diagramme de séquence de la réservation

La Gateway vérifie le jeton du patient. MS4 appelle MS2 (le médicament existe-t-il ?), MS1 (quelle pharmacie ?), puis MS3 qui décrémente le stock de manière atomique. Si MS3 répond `409`, rien n'est créé. Sinon MS4 enregistre la réservation avec un code de retrait, répond `201` et publie `reservation.events` : MS5 notifie le patient et le pharmacien sans que le patient attende.

5.5 Scénario asynchrone : alerte de retour en stock

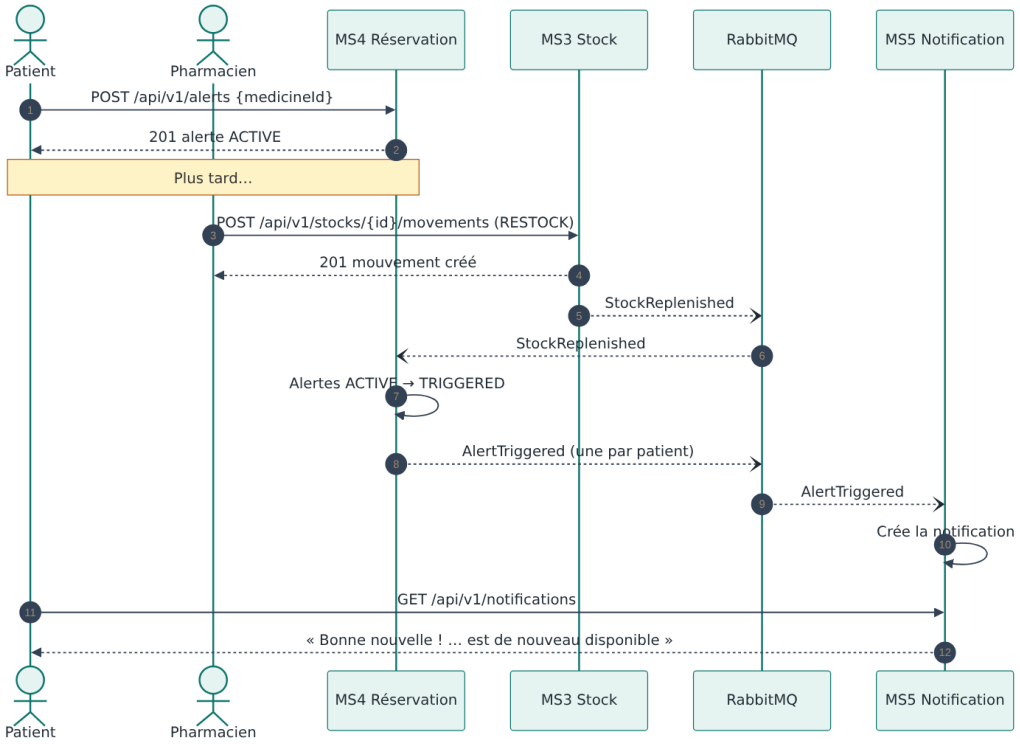


Figure 5.2 – Diagramme de séquence de l'alerte

Le patient crée une alerte (MS4). Quand le pharmacien réapprovisionne (MS3), MS3 publie `stock.replenished` sans savoir qui l'écoute. MS4 déclenche les alertes actives pour ce médicament et publie `alert.triggered` ; MS5 crée la notification. Si MS4 est arrêté, le message attend dans la file et sera traité au redémarrage (cohérence à terme).

5.6 Cohérence et gestion des pannes

— **Pas de survente** : MS3 exécute `UPDATE stock SET quantity = quantity - :q WHERE id = :id AND quantity >= :q` ; zéro ligne modifiée signifie stock insuffisant (`409`).

— **Compensation (saga)** : si MS4 échoue à enregistrer la réservation après le décrément, ou lorsqu'une réservation est annulée ou expirée, MS4 publie `reservation.released` et MS3 rend les unités.

— **Expiration** : une tâche planifiée de MS4 passe les réservations non retirées à `EXPIRED`.

— **Circuit breaker** : après plusieurs échecs, l'appel n'est plus tenté pendant un délai et la méthode de repli (*fallback*) est utilisée.

— **Idempotence** : chaque événement porte un identifiant ; un consommateur ignore un message déjà traité.

5.7 Conventions des API

— URI en noms au pluriel, versionnées : `/api/v1/pharmacies`, filtres dans la chaîne de requête (`?onDuty=true`, `?medicineId=9`), pagination `?page=&size=`.

— Verbes : `GET` lecture, `POST` création (`201` + en-tête `Location`), `PUT` remplacement, `PATCH` modification partielle, `DELETE` suppression (`204`).

— Codes d'erreur : `400` données invalides, `401` non authentifié, `403` interdit, `404` introuvable, `409` conflit métier, `503` dépendance indisponible.

— Erreurs au format *Problem Details* (RFC 9457) :

```
{
  "type": "https://medfinder.local/errors/insufficient-stock",
  "title": "Stock insuffisant",
  "status": 409,
  "detail": "Only 1 unit left for medicine 9 in pharmacy 1",
  "instance": "/api/v1/reservations"
}
```

CHAPITRE 6

Conception détaillée des microservices

6.1 MS1 · Pharmacies

Responsable : Jihed Khlifi · **Base :** H2 · **Port :** 8081

6.1.1 Responsabilités

Gérer les pharmacies (adresse, horaires, coordonnées GPS), les pharmacies de garde, la recherche par distance et la validation des pharmacies par l'administrateur.

6.1.2 Diagramme de classes

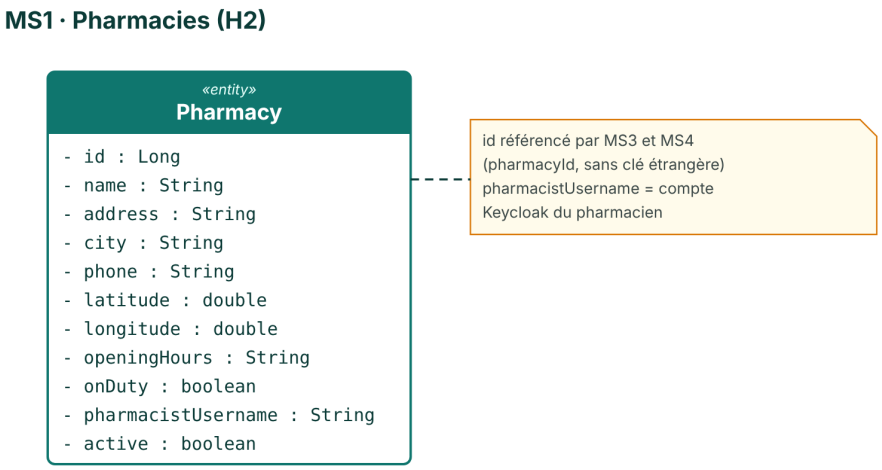


Figure 6.1 – MS1 – diagramme de classes

6.1.3 Interactions

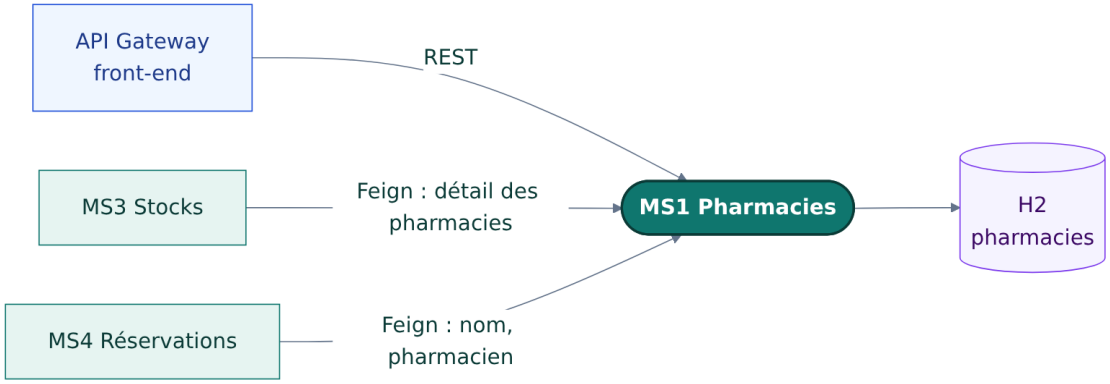


Figure 6.2 – MS1 – interactions

— **Synchrone :** appelé par MS3 et MS4 (OpenFeign) pour lire une pharmacie ; n'appelle aucun service.

— **Asynchrone :** aucun événement (service de référence).

— **En cas de panne :** MS3 et MS4 continuent en mode dégradé, sans le nom de la pharmacie.

6.1.4 Contrat d'API

Verbe	Endpoint	Action · accès	Codes
GET	`/api/v1/pharmacies?onDuty=true`	Pharmacies de garde · public	200
GET	`/api/v1/pharmacies?lat=&lng=`	Pharmacies proches · public	200
GET	`/api/v1/pharmacies/{id}`	Détail · public	200, 404
POST	`/api/v1/pharmacies`	Créer · ADMIN	201, 400, 403
PUT	`/api/v1/pharmacies/{id}`	Remplacer · ADMIN, pharmacien propriétaire	200, 403, 404
PATCH	`/api/v1/pharmacies/{id}`	Garde, validation · ADMIN, propriétaire	200, 403, 404
DEL	`/api/v1/pharmacies/{id}`	Supprimer · ADMIN	204, 404

Événements : aucun publié, aucun consommé.

6.2 MS2 · Catalogue des médicaments

Responsable : Abdelhamid Ayadi · **Base :** H2 · **Port :** 8082

6.2.1 Responsabilités

Gérer les médicaments et leurs catégories, la recherche par nom ou molécule active, les génériques (même molécule) et l'indication d'ordonnance.

6.2.2 Diagramme de classes

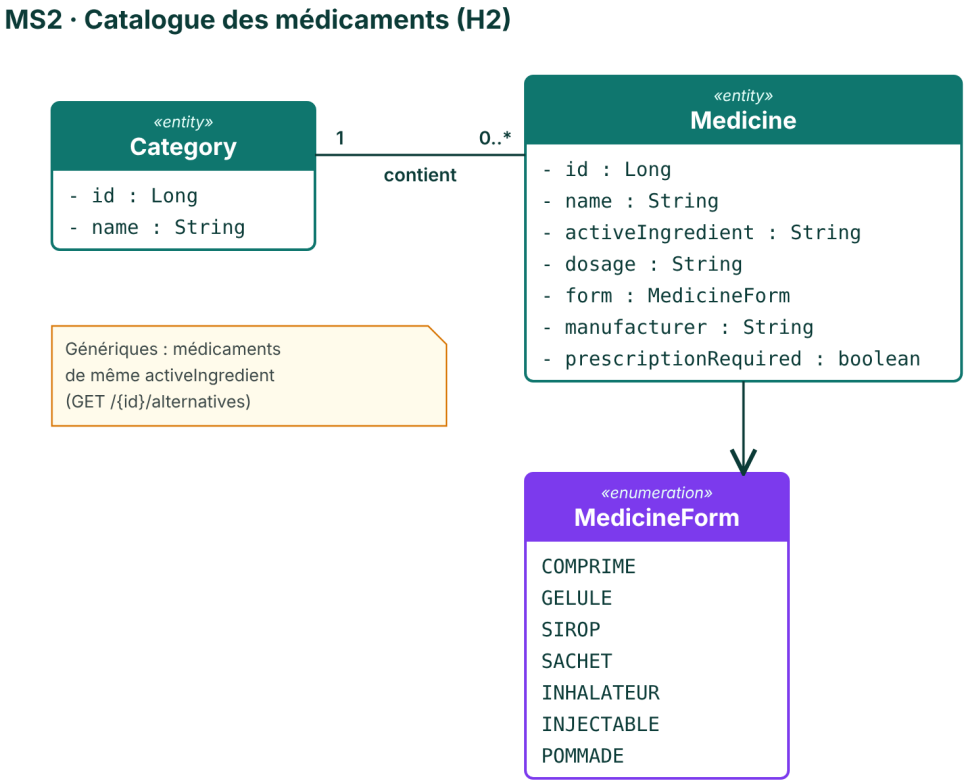


Figure 6.3 – MS2 – diagramme de classes

6.2.3 Interactions

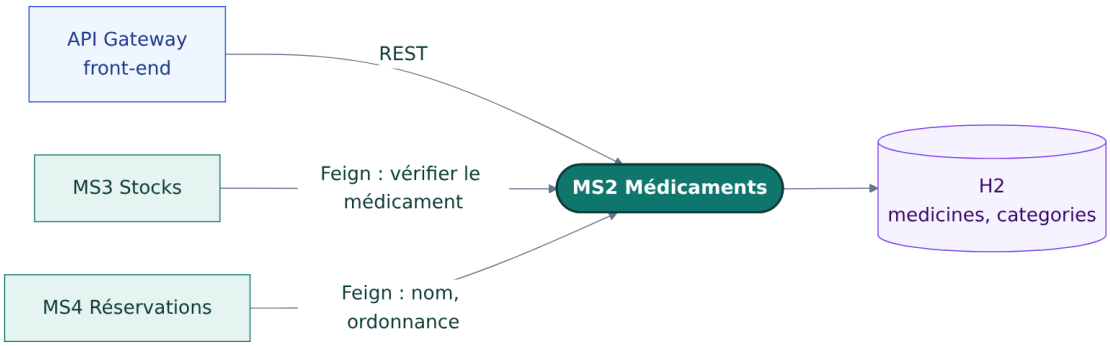


Figure 6.4 – MS2 – interactions

— **Synchrone :** appelé par MS3 (vérifier un médicament avant l'ajout au stock) et MS4 (nom, ordonnance).

— **Asynchrone :** aucun événement (service de référence).

— **En cas de panne :** ajout au stock et réservation refusés avec une erreur `503` claire, plutôt que des données incohérentes.

6.2.4 Contrat d'API

Verbe	Endpoint	Action · accès	Codes
GET	`/api/v1/medicines?q=&page=`	Rechercher · public	200
GET	`/api/v1/medicines/{id}`	Détail · public	200, 404
GET	`/api/v1/medicines/{id}/alternatives`	Génériques · public	200, 404
POST	`/api/v1/medicines`	Créer · ADMIN	201, 400, 409
PUT	`/api/v1/medicines/{id}`	Modifier · ADMIN	200, 404
DEL	`/api/v1/medicines/{id}`	Supprimer · ADMIN	204, 404
GET	`/api/v1/categories`	Lister · public	200
POST	`/api/v1/categories`	Créer · ADMIN	201, 409

Événements : aucun publié, aucun consommé.

6.3 MS3 · Stocks

Responsable : Ferdaouas Oueslati · **Base :** H2 · **Port :** 8083

6.3.1 Responsabilités

Gérer le stock de chaque médicament dans chaque pharmacie (quantité, prix, seuil bas), répondre à « qui a ce médicament ? », enregistrer réapprovisionnements et historique, et réserver des unités sans survente.

6.3.2 Diagramme de classes

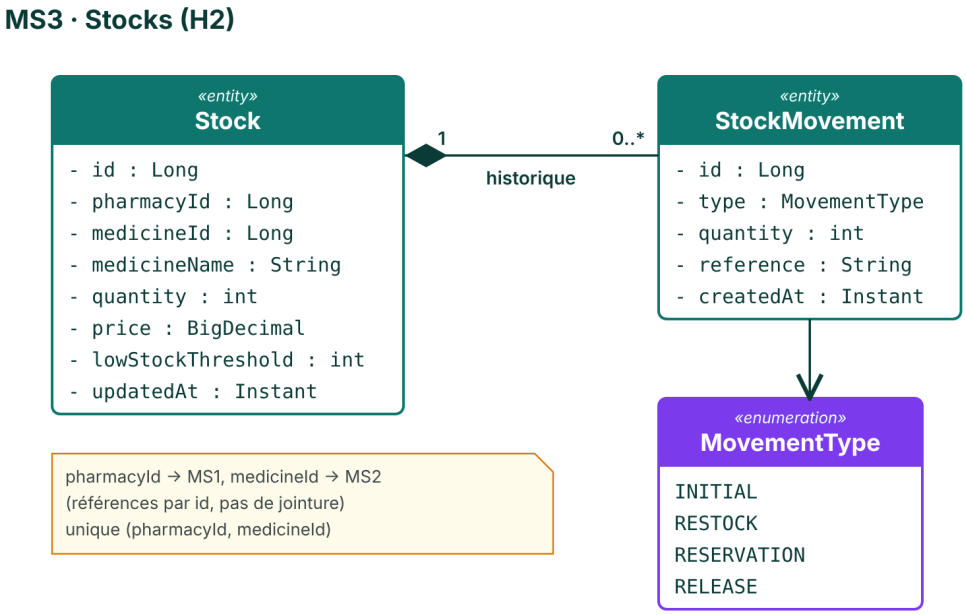


Figure 6.5 – MS3 – diagramme de classes

6.3.3 Interactions

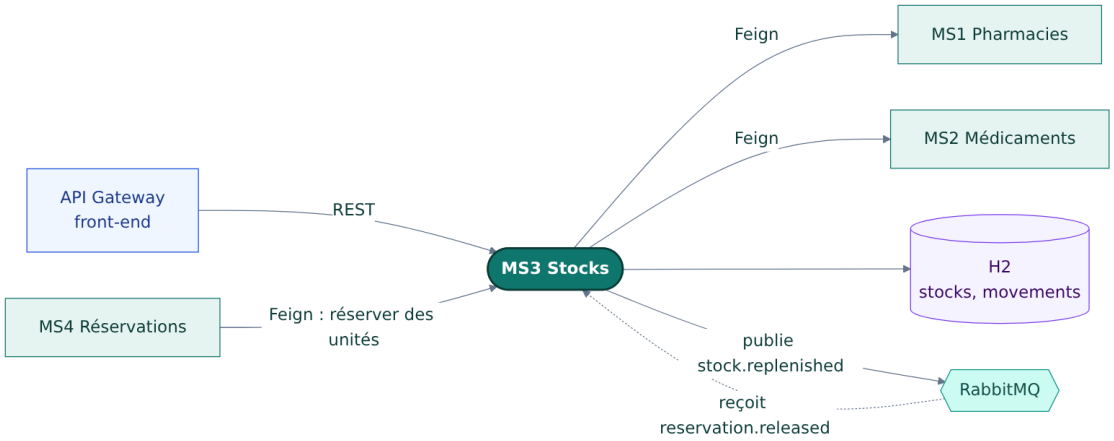


Figure 6.6 – MS3 – interactions

— **Synchrone :** appelle MS1 (détail des pharmacies) et MS2 (vérification du médicament) ; appelé par MS4 pour réserver des unités.

— **Asynchrone :** publie `stock.replenished` ; consomme `reservation.released` pour rendre les unités.

— **En cas de panne :** si MS1 tombe, stocks affichés sans le détail des pharmacies ; si MS2 tombe, ajout au stock refusé (`503`).

6.3.4 Contrat d'API

Verbe	Endpoint	Action · accès	Codes
GET	`/api/v1/stocks?medicineId=9`	Pharmacies qui ont le médicament · public	200
GET	`/api/v1/stocks?pharmacyId=1`	Stock d'une pharmacie · public	200
POST	`/api/v1/stocks`	Ajouter un médicament au stock · PHARMACIST	201, 403, 409
PATCH	`/api/v1/stocks/{id}`	Ajuster quantité, prix, seuil · PHARMACIST	200, 403, 404
POST	`/api/v1/stocks/{id}/movements`	Réapprovisionner (mouvement) · PHARMACIST	201, 403, 404
DEL	`/api/v1/stocks/{id}`	Retirer du stock · PHARMACIST	204, 403
POST	`/api/v1/stocks/internal/reservations`	Prendre des unités · interne (MS4)	201, 409

Événements : publie `stock.replenished` ; consomme `reservation.released`.

6.4 MS4 · Réservations et alertes

Responsable : Insaf Makhloufi · **Base :** MySQL · **Port :** 8084

6.4.1 Responsabilités

Créer les réservations et leur code de retrait, suivre leur état (en attente, prête, retirée, annulée, expirée), gérer l'expiration automatique et les alertes de retour en stock.

6.4.2 Diagramme de classes

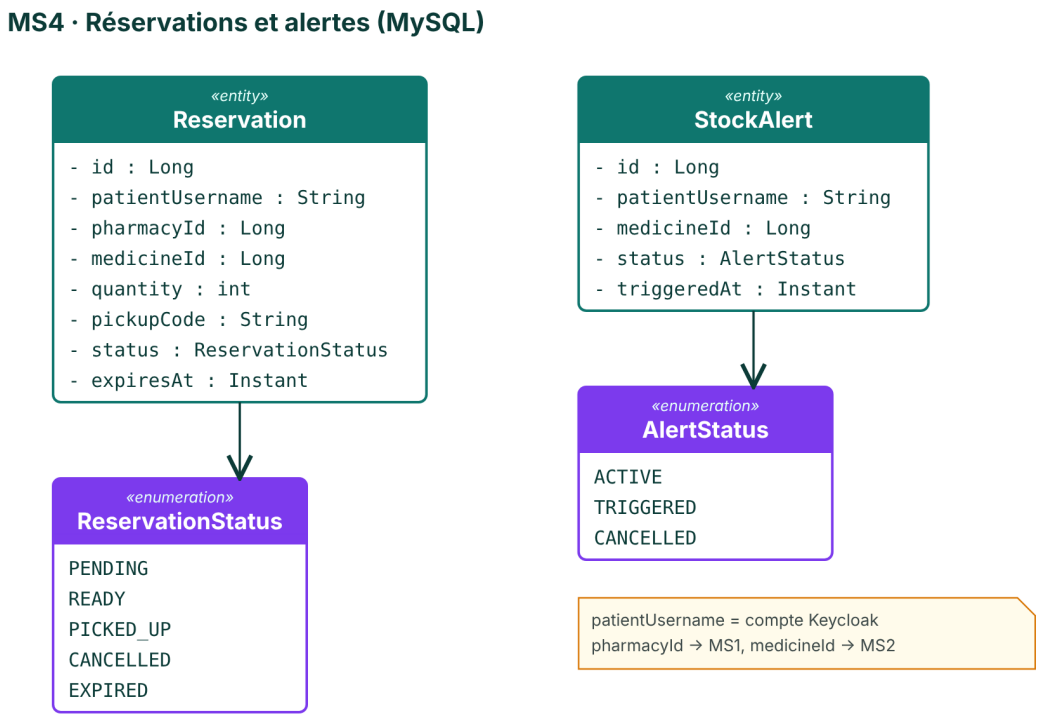


Figure 6.7 – MS4 – diagramme de classes

6.4.3 Interactions

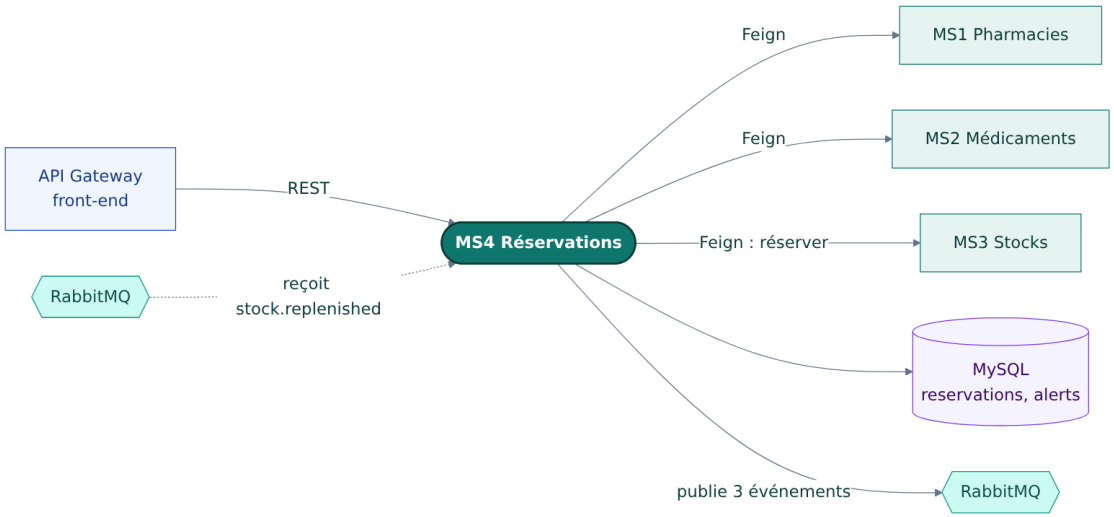


Figure 6.8 – MS4 – interactions

— **Synchrone :** appelle MS2 (médicament), MS1 (pharmacie) puis MS3 (prise des unités) pour créer une réservation.

— **Asynchrone :** consomme `stock.replenished` (déclenche les alertes) ; publie `reservation.events`, `alert.triggered`, `reservation.released`.

— **En cas de panne :** si MS3 tombe, réservation refusée (`503`) et rien n'est enregistré ; si MS1 tombe, réservation créée sans le nom de la pharmacie.

6.4.4 Contrat d'API

Verbe	Endpoint	Action · accès	Codes
POST	`/api/v1/reservations`	Réserver · PATIENT	201, 409, 503
GET	`/api/v1/reservations?status=`	Mes réservations (patient) ou celles de ma pharmacie (pharmacien)	200
GET	`/api/v1/reservations/{id}`	Détail · propriétaire	200, 403, 404
PATCH	`/api/v1/reservations/{id}`	Annuler (patient), prête / retirée (pharmacien)	200, 403, 409
POST	`/api/v1/alerts`	Créer une alerte · PATIENT	201, 409
GET	`/api/v1/alerts`	Mes alertes · PATIENT	200
DEL	`/api/v1/alerts/{id}`	Supprimer une alerte · PATIENT	204, 404

Événements : publie `reservation.events`, `alert.triggered`, `reservation.released` ; consomme `stock.replenished`.

6.5 MS5 · Notifications et statistiques (microservice commun)

Responsable : toute l'équipe (pilotage : Ferdaouas Oueslati) · **Base :** MongoDB · **Port :** 8085

6.5.1 Responsabilités

Écouter les événements métier, créer les notifications des patients et des pharmaciens, journaliser tous les événements et calculer les statistiques de l'administrateur.

6.5.2 Diagramme de classes

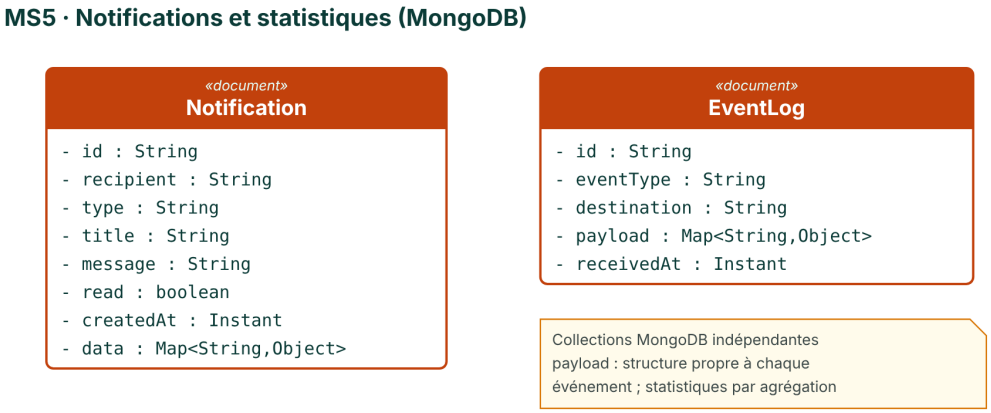


Figure 6.9 – MS5 – documents MongoDB

6.5.3 Interactions

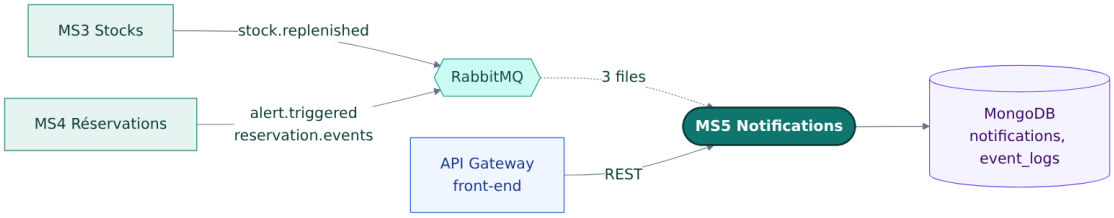


Figure 6.10 – MS5 – interactions

— **Synchrone :** aucun appel sortant ; appelé uniquement via la Gateway par le front-end.

— **Asynchrone :** consomme `stock.replenished`, `alert.triggered` et `reservation.events`.

— **En cas de panne :** les messages attendent dans les files RabbitMQ et sont traités au redémarrage ; aucun autre service n'est bloqué.

6.5.4 Contrat d'API

Verbe	Endpoint	Action · accès	Codes
GET	`/api/v1/notifications?read=false`	Mes notifications · authentifié	200, 401
PATCH	`/api/v1/notifications/{id}`	Marquer comme lue · destinataire	200, 403, 404
PATCH	`/api/v1/notifications`	Tout marquer comme lu · authentifié	204
GET	`/api/v1/events?type=`	Journal des événements · ADMIN	200, 403
GET	`/api/v1/statistics`	Statistiques · ADMIN	200, 403

Événements : aucun publié ; consomme les trois files ci-dessus.

CHAPITRE 7

Organisation du projet

7.1 Répartition des tâches

Membre	Microservice	Partie commune	Partie présentée
Jihed Khlifi	MS1 Pharmacies	API Gateway, Swagger	Problématique, solution, acteurs
Abdelhamid Ayadi	MS2 Médicaments	Keycloak et sécurité	Microservices, architecture, communication
Ferdaouas Oueslati	MS3 Stocks	Docker, RabbitMQ, pilotage MS5	Exemples de communication, MS5
Insaf Makhloufi	MS4 Réservations	Eureka, Config Server	Technologies, répartition, Git, planning

Table 7.1 – Répartition des tâches

Le microservice commun MS5 est développé par tous ; chaque membre développe aussi les pages du front-end liées à son microservice.

7.2 Dépôt Git

Dépôt public : `https://github.com/insafmakhloufi/projweb-medfinder-4DASI11` (Mme Ons Ben Salah est ajoutée comme collaboratrice).

```
projweb-medfinder-4DASI11/
|-- Backend/
|   |-- config-server/            Spring Cloud Config Server
|   |-- eureka-server/            registre des services
|   |-- api-gateway/              point d'entree unique
|   |-- pharmacy-service/         MS1 Pharmacies
|   |-- medicine-service/         MS2 Medicaments
|   |-- stock-service/            MS3 Stocks
|   |-- reservation-service/      MS4 Reservations et alertes
|   |-- notification-service/     MS5 Notifications (commun)
|-- Documentation/
|   |-- Diagrams/                 global + un dossier par microservice
|   |-- Presentation/             support de la validation (PDF)
|   |-- Dossier_technique_MedFinder.pdf
|-- Frontend/                     application React
|-- README.md                     sujet, membres, architecture, lancement
```

— Branche `main` protégée ; une branche par fonctionnalité (`feature/ms3-stock-crud`) ; fusion par *pull request* relue par un autre membre.

— Messages de commit normalisés : `feat(ms4): ...`, `fix(gateway): ...`, `docs: ...`.

— Chaque membre committe lui-même son service et sa partie commune : l'historique reflète le

travail de chacun.

7.3 Gestion de projet

Un tableau GitHub Projects (colonnes *À faire*, *En cours*, *En revue*, *Terminé*) contient le backlog de 42 tâches, chacune associée à un membre, un microservice et une semaine. Les *issues* sont liées aux *pull requests*. Un point d'équipe hebdomadaire fait le bilan et réaffecte les tâches en retard.

7.4 Planning

Semaine	Objectif
S4	Validation de la conception ; projets créés dans le dépôt
S5	Entités, CRUD et contrats d'API de chaque microservice ; Gateway, Eureka, Config Server
S6	Communication synchrone (OpenFeign) et circuit breaker
S7	RabbitMQ, événements et microservice commun MS5
S8	Sécurité Keycloak de bout en bout
S9	Conteneurisation Docker et front-end React
S10	Tests, finitions et démonstration finale

7.5 Risques identifiés

Risque	Mesure prévue
Configuration Keycloak complexe	Royaume exporté dans le dépôt et importé automatiquement au démarrage
Incohérence de stock (survente)	Décrément atomique et compensation par événement
Service indisponible	Circuit breaker, mode dégradé, files RabbitMQ durables
Machines limitées en mémoire	Limites mémoire par conteneur ; lancement partiel par profil
Retard d'un membre	Backlog partagé, revues croisées, réaffectation en point d'équipe
Intégration tardive	Contrats d'API figés dès S5, intégration continue chaque semaine

Conclusion

Ce dossier présente la conception de MedFinder soumise à la validation intermédiaire : cinq microservices issus des contextes bornés du domaine, chacun propriétaire de sa base, une infrastructure Spring Cloud complète (Gateway, Eureka, Config Server), une sécurité centralisée avec Keycloak et une communication mixte, synchrone lorsque la réponse est nécessaire et asynchrone pour diffuser les faits métier. Les comportements en cas de panne et les mécanismes de cohérence sont définis dès maintenant. Les projets sont créés dans le dépôt Git ; la logique métier sera implémentée à partir de la semaine 5 selon le planning présenté.

ANNEXE A

Exemples d'échanges

A.1 Créer une réservation

Requête `POST /api/v1/reservations` (jeton du patient) :

```
{ "pharmacyId": 1, "medicineId": 9, "quantity": 2 }
```

Réponse `201 Created`, en-tête `Location: /api/v1/reservations/42` :

```
{
  "id": 42, "patientUsername": "patient1", "pharmacyId": 1,
  "pharmacyName": "Pharmacie␣Centrale", "medicineId": 9,
  "quantity": 2, "pickupCode": "MF-7K3Q9", "status": "PENDING",
  "expiresAt": "2026-11-26T10:15:00Z"
}
```

A.2 Événement `stock.replenished`

```
{
  "eventId": "6f1c2e0a-...", "type": "stock.replenished",
  "stockId": 17, "pharmacyId": 1, "medicineId": 9,
  "medicineName": "Doliprane␣1000␣mg", "quantity": 30,
  "occurredAt": "2026-11-25T08:00:00Z"
}
```

A.3 Événement `reservation.released`

```
{
  "eventId": "a93d...", "type": "reservation.released",
  "reservationId": 42, "pharmacyId": 1, "medicineId": 9,
  "quantity": 2, "reason": "EXPIRED"
}
```